



Open ASL Translation and Compounding Public Models

Public whitepaper and successor mechanism specification

Protocol version	0.2
Network	Bittensor mainnet, subnet 78
Status	Successor specification and implementation work; open competition rewards inactive; temporary registration-bridge weights governed by a separate signed policy and fixed sunset
Publication date	September 2026

Successor design; activation requires a separately signed policy.

Contents

1	Scope and transition	1
2	Participation and identity	1
3	Two tracks	2
3.1	Endpoint service	2
3.2	Model contribution	2
4	Evaluation	3
4.1	Data and holdouts	3
4.2	Deterministic quality	3
4.3	Evaluators and trust	4
4.4	Untrusted execution	5
5	Preservation and baseline promotion	5
6	Licenses, ownership and product use	6
7	Reward calculation	6
8	Policy and state boundaries	7
9	Threats and limitations	7
10	Release and activation	7
11	Implementation status	8
	References	8

Abstract

UMI develops ASL-to-English translation through two complementary forms of competition. Endpoint miners serve translations using systems they control. Model contributors submit reproducible, licensed model bundles that UMI can preserve and operate independently. Both are evaluated against held-out ASL examples. Verified improvements can become the next public reference model.

The network must produce useful translations and an accumulating body of runnable models. Access to a successful endpoint alone cannot meet the second objective. Model promotion therefore requires possession of the model files, a reproducible inference environment, documented rights, and verification without the original miner or an external inference API.

The first successor release separates implementation rehearsal from reward activation. No existing miner must disclose proprietary weights retroactively. The private endpoint track remains available under its published rules; the model-contribution track has explicit publication and reuse terms.

1 Scope and transition

The task is raw American Sign Language video to bounded English text. A miner may use an end-to-end video model, pose model, ensemble or hosted API for endpoint service, subject to the service deadline and applicable terms. A promoted model must run with its archived dependencies and no external inference service. Future tasks require their own evaluation and reward policy.

The frozen two-miner pilot bootstrap is retired. Its signed manifest and historical evidence remain unchanged; see the [bootstrap addendum](#) and [shared-validator supersession](#). It was replaced by the separately signed [temporary registration bridge](#). The [September 13 deployment record](#) records funding-grouped bridge rows finalized by validators UID 0 and UID 54.

The bridge checks registered miners' chain-announced HTTPS health endpoints. Prior pilot participation and a running translation model are not required. Passing miners connected by a shared coldkey, HTTPS endpoint IP or recorded pre-registration funding source share one equal group weight budget, divided among their passing UIDs. Validator-permitted and owner-associated identities remain excluded. The [miner guide](#) gives the complete eligibility and endpoint requirements.

Funding assertions bind an exact registration identity in the signed policy. A cached coordinator checker detects new registrations; new funding links affect weights only after a signed snapshot refresh. Validators and miners do not each need a Taostats API key. Shared-sender grouping can combine independent exchange customers, and separate funding sources can evade it. It does not prove common human ownership. The [funding-cap rule](#) is temporary and is not a requirement of the 70/30 successor mechanism specified below.

Bridge weights measure availability only. They establish neither translation quality nor successor qualification. A separately signed version3 bridge policy can continue until explicitly superseded, with no scheduled expiry. Historical version1/2 policies retain their original cutoffs. A new lifetime authorization and compatible release are required; this document alone does not activate an extension, reopen the retired pilot or launch translation competition.

Version 0.2 supersedes [version 0.1](#)'s proposed fixed four-validator, three-publisher and 30-day-soak launch gates. It replaces them with a published evaluation cohort, disclosed control groups, bounded rehearsals, policy-selected result reproduction and the explicit activation checklist in Section 10. This change is a UMI policy revision. It is not evidence that any earlier activation gate was completed.

The active mechanism and policy hash must be distinguishable from proposed rules in the website, observer API, miner guide and signed validator feed. Software installation, a successful test, and positive native chain emissions are not interchangeable with activation of this competition.

2 Participation and identity

A publisher prepares and publishes challenge batches under the data policy. A validator's supervisor is the host service that checks signed release directives and manages its worker. The worker image is the versioned container containing the validator task code.

Anyone with a registered SN78 hotkey can submit through the same public interface. A prior pilot, invitation, GitHub issue or private upload key is not an admission requirement. Registration proves control of an identity; it does not establish translation quality or a right to rewards.

A submission binds:

- network, subnet, competition policy hash and track;
- the submitting hotkey and a monotonic submission sequence;
- the model or endpoint revision and applicable publication terms;
- the validity interval in finalized blocks; and
- the complete artifact manifest for a model contribution.

The hotkey signs a domain-separated digest of canonical JSON. Signatures from another network, policy, identity or track cannot be reused. The registry returns the same receipt for an identical retry. A conflicting sequence is rejected. The policy sets the minimum interval between accepted replacements and admits at most one current submission per hotkey per track.

The registry publishes accepted submissions and explicit rejection reasons. Pending work survives process restarts. A UID is resolved from the current finalized hotkey mapping at the scoring boundary and rechecked before a weight submission; UID reuse must not transfer another miner's score. Deregistration, permit changes and revision replacements cannot rewrite a closed evaluation roster.

The coordinator must publish the complete admission log and each round's roster. Queue receipt time, accepted block and expiry remain distinct. A delayed scheduler cannot backdate a usable evaluation window or count an unissued request as a miner failure. Closed rounds are immutable. Capacity limits and the deterministic selection rule must be public.

3 Two tracks

3.1 Endpoint service

An endpoint miner runs the UMI translation protocol using its own model or provider. The miner retains its weights and implementation unless it separately opts into model contribution. A health-only keepalive earns no translation score.

Each accepted endpoint revision is challenged within a published request budget. Requests and answers are authenticated and bound to the round, clip, policy and revision. Raw video is verified before use. A miner timeout or invalid response scores zero for that assigned case. Coordinator, publisher or evaluator infrastructure failure voids the affected evaluation instead of being charged to the miner.

An unavailable miner does not stall the entire cohort. The next complete round calculates its row from valid evidence and public failure rules. Retries cannot replace an observed answer with a more favorable answer.

3.2 Model contribution

A model contributor publishes a versioned bundle under the policy's accepted reuse terms. Its content identity is independent of the submitting UID, repository name and upload time. The same bytes submitted under another hotkey do not become a new model.

A complete contribution contains:

- weights, including the exact base weights needed by an adapter or LoRA;
- architecture and inference configuration;
- tokenizer, vocabulary, processor and all preprocessing assets;
- inference code and a pinned environment;
- license texts, attribution, upstream dependency identities and provenance;
- the parent public baseline identity, if derived from it; and
- instructions for a clean, offline inference run.

A repository URL or a hash-only declaration is insufficient for promotion. The archive verifier reads every declared byte and rejects missing, extra, oversized or changed files, path traversal, links and special files. External paid inference and a dependency on the original miner's server disqualify the bundle from the offline promotion profile.

Automatic admission validates structure, signatures and declared terms. It does not prove authorship, consent or compatibility of upstream licenses. Promotion includes review of those rights. Review happens on the submitted bundle; enrollment must not require private credentials or wallet material.

4 Evaluation

4.1 Data and holdouts

The launch uses automatic scoring against a private, labeled ASL holdout. It does not require a person to grade miner outputs or a new ASL review panel before activation. Existing dataset annotations may supply the English references when their source, annotation procedure and permitted use are documented. Dataset validation and model-contribution rights review remain required; neither is a per-output human scoring step.

Reserve the evaluation split before training or tuning. Clips, labels and near-duplicate recordings used to train or select the baseline or a candidate cannot count as unseen evaluation evidence for that model. Making training data private afterward does not make it a holdout. Record known training overlap and unknown upstream exposure rather than claiming they have been ruled out. Publish a development set separately. Hidden evaluation clips and labels must not become miner training data through repeated tests, debug logs, image layers or unrestricted evaluator APIs.

A round commits its exact policy, candidate roster, incumbent baseline, evaluation suite and runtime profile. Candidates and incumbent are scored against the same suite. A score from a different suite or baseline cannot justify promotion. Suites are retired after their protected evaluation interval and replaced before adaptive resubmissions can exploit them.

Evaluation evidence binds video hashes, accepted references, outputs, failures, timing, environment and evaluator identities. The public replay package reveals only material whose publication is permitted by its consent and data policy. When underlying clips cannot be redistributed, disclose that limitation and retain access-controlled independent verification; do not call a hash alone public reproducibility.

Data collection and labeling can be developed separately. They receive no implicit share of this policy's miner rewards. The [private-holdout launch procedure](#) describes required inputs, annotation checks, split isolation and suite retirement.

4.2 Deterministic quality

Reuse UMI's exact text normalization and CER/WER implementation. The approved initial launch uses `umi-open-competition-policy/2` with `umi-competition-suite/2`: each case has exactly one authentic committed English reference. Historical version 1 keeps its three-to-five-reference contract. CER measures edit distance between normalized graphemes, excluding whitespace; WER uses normalized word tokens. For each reference, let d be the edit distance and n its number of scoring units. Its similarity is $\max(0, 1 - d / \max(1, n))$. The case score is the highest similarity across the committed references.

The initial version 2 profile uses CER for fingerspelling with exact weight $3/13$, and WER for continuous signing with exact weight $10/13$. These preserve the original 15:50 relative weights across the two available tasks. Short utterances are outside this launch profile. Version 1 retains its 15%/35%/50% three-task weights. Compute the arithmetic mean of the case scores within each stratum, then sum those means multiplied by their respective weights. Every required stratum must meet the policy's minimum case count. Suites contain at least three cases in total. Policy and suite versions must match; unknown or extra strata are rejected. The two-task score is not directly comparable to a historical three-task score. This change does not alter the 70/30 reward split or the approved 120-second inference limit. The normalization and scoring functions are defined in [scoring.py](#).

Record integer case errors and exact rational aggregates. Floating-point rounding cannot determine a promotion or a tie. The policy declares:

- minimum qualifying quality;
- the absolute improvement margin for promotion;

- the inference time and output-size limits;
- the number of cases per stratum and evaluator control-group quorum; and
- submission rate, artifact-size and queue bounds.

A promoted candidate must improve the aggregate by at least the declared margin and must not regress in any required stratum. It must pass the offline resource and reconstruction checks as well. A tie retains the incumbent. No score is a claim of clinical fitness or a replacement for product-specific safety evaluation. CER/WER measure agreement with the committed English references. Without separate semantic validation, report these as automatic benchmark scores, not human-verified ASL understanding. Valid paraphrases and meaning-changing errors can be misranked by text edit distance.

4.3 Evaluators and trust

A policy lists evaluator hotkeys and their actual administrative control groups. Evaluators are automated execution and signing services; their operators do not need to grade ASL outputs. Multiple hotkeys controlled by one operator count as one group. UID 0 and UID 54 under the same administration cannot supply two independent votes.

The approved initial launch selects only UID 0, hotkey 5Fk765B4CRBekwErwE5VxvveWhHztHSfsnsLt8cbDayDWSuk, in one disclosed UMI-operated control group, with `required_evaluator_groups=1`. This initial phase trusts one operator's automated evaluation and publication; it does not provide independent cross-operator reproduction or resistance to that operator's collusion. UID 54 is not a second evaluator or vote. Adding evaluators or raising the quorum requires a later explicit signed policy and deployment rehearsal. See the [UID 0 launch profile](#).

Evaluators sign the same canonical evaluation result. A quorum certificate contains valid signatures from the policy's required number of distinct control groups. Multiple keys from one group cannot increase that count. A small disclosed cohort is a trust assumption, not permissionless evaluation.

The current rehearsal schema represents one common recorded transcript: candidate and incumbent outputs, outcome statuses, elapsed milliseconds, completion block and the pinned runtime identity. All of those fields are inside the signed result. A co-signature endorses that transcript; it does not claim that the signer measured identical wall-clock timing in a separate run. The result object remains unchanged. A separate evidence envelope now binds one independently signed run record to each common-result signer. Each record binds the policy, round, submission, common result, suite, model and runtime, with its own interval, timings and execution-evidence digest. Outputs and per-case resource eligibility must agree with the common transcript, and exact score replay must agree. The run signer set must match the common certificate with one key per authorized control group. This contract has local tests; production execution and evidence retention still require rehearsal. In the initial single-operator phase there is one signed run record, not an independent second run. Later multi-group policies require each selected group to reproduce the result. Co-signatures and execution-evidence digests alone do not prove a run occurred.

The local paired-model runner now records bounded raw outputs and finalized boundary references in a separate wallet-free execution journal. It reserves an attempt before running model code, retains returned outputs before the next finality read and refuses automatic reruns after interruption. Completed retries return the original evidence. Local aggregation checks the revealed suite and distinct evaluator groups, proposes the maximum observed timing per case, and prepares unsigned run records. This does not supply production round publication, independent operators or authorization to submit weights.

The evidence identity is the policy, round and miner-submission digest. Store each authenticated quorum result and its signatures durably. Reordering signatures or adding signatures for the same result is an idempotent retry. Two different quorum-certified results for one evidence identity put the entire round on conflict hold, even when the quorums have no group in common. Preserve both results and any proof that a control group signed both. Another hotkey in that group cannot erase its prior statement.

Invalid signatures, incorrect bindings and a statement without the required quorum cannot trigger this automatic hold. Ordinary non-quorum disagreement does not give a minority a veto. Any separate group-exclusion rule requires an explicit policy; it cannot be invented while settling a disputed round.

Authentication checks the historical assignment and result interval, separate from present eligibility or whether the result earns a score. A valid failed outcome, expired reward window or failed archive check cannot erase evidence of conflicting statements. Commit that evidence before reporting a rejected action. Promotion and reward projection read the recorded results and check the round's conflict state within their action transaction, including retries.

Missing quorum evidence prevents settlement. Before activation, the policy must fix the evidence-collection cutoff and the procedure for publishing an immutable settlement record. A local projection cannot establish that all evidence has arrived. The local store fixes a cutoff before round close, retains first-observation blocks and settles only evidence recorded by that cutoff. Its immutable artifact binds the roster, results, run evidence, suite, registration snapshot, promotion head and projected row. Local block inputs and records are not finalized observations or signed publication authority. Late conflicting certificates remain admissible as incident evidence and stop further use of the disputed round. They cannot rewrite historical records or undo payments already finalized on chain.

Retain requests and outputs so a verifier can recompute scores. Inference reproduction is distinct from arithmetic replay: hardware-specific numerical variation must be investigated under a pinned runtime profile rather than hidden behind approximate score agreement.

4.4 Untrusted execution

Downloaded artifacts never execute inside a wallet-bearing validator process. Use a separate, resource-limited evaluation environment with no wallet mounts, host sockets, private keys, cloud credentials or plaintext reference labels. Allow only the assigned public inputs and a bounded output channel.

The offline profile disables network access, uses read-only model files, pins the evaluator runtime, and enforces process, CPU, memory, storage, output and wall-clock limits. A timeout terminates the complete workload. Merely starting a container does not prove these controls are effective; release tests must demonstrate them on the supported host.

Large VLMs may require GPU evaluators. Existing CPU-only validator hosts are not promised the ability to run arbitrary contributed models. Each activated policy pins a supported evaluation budget. Training hardware remains the miner's choice.

5 Preservation and baseline promotion

A baseline registry is an append-only sequence of promotions. Each entry binds the previous baseline, new model manifest, policy, evaluation result, agreeing evaluator signatures, archive verification and rights review. The initial baseline is imported explicitly with its own manifest and provenance. The imported initial baseline has no registered contributor and earns no model-contribution share. Until the first qualifying promotion, a policy must allocate zero basis points to that track or no successor reward row can be produced.

Promotion follows this order:

1. Verify the submitted identity and complete candidate bundle.
2. Reconstruct and run the model offline within the declared resource budget.
3. Replay the paired evaluation and verify evaluator-group agreement.
4. Verify rights and archive the exact files in UMI-controlled storage.
5. Commit a compare-and-swap update against the evaluated incumbent.
6. Publish the new baseline and its promotion evidence.

If another promotion changed the incumbent, the candidate needs a new paired evaluation. An old favorable result cannot overwrite a newer baseline. An interrupted archive or failed verification leaves the prior baseline unchanged. Repeating the same successful promotion is idempotent. That retry remains subject to the conflict guard. If late evidence disputes a completed promotion, preserve its record and archived files, but hold further promotion and reward projection using that baseline or its descendants. Resuming requires an explicit recovery transition; a newer round alone cannot clear the hold or silently reassign the original contribution.

A promoted version remains retrievable when the miner disappears or changes its repository. Maintain a separate backup and periodically test restoration. Hugging Face is a distribution channel; a mutable branch or a single external repository is not the preservation record.

Future miners may download and improve the current baseline. They must retain required attribution and identify their parent version. Cosmetic repackaging or a copied checkpoint cannot earn a fresh promotion reward. More general model-copy detection is imperfect and must not be presented as cryptographic proof of independent training.

6 Licenses, ownership and product use

Publication grants only the rights supplied by the applicable licenses. It does not assign exclusive intellectual-property ownership to UMI. Require rights sufficient for preservation, commercial inference, modification and redistribution, with compatible upstream obligations. Exclusive acquisition, when desired, needs a separate agreement.

The current reference repository labels its code Apache-2.0 and its weights and portable bundle CC BY-SA 4.0. Those obligations are not erased when a miner fine-tunes, changes hosting, or submits a model. Preserve license text, attribution and provenance with every promoted version. Do not silently relabel inherited weights under a different license.

A contributor must identify base-model and dataset sources and attest that it can grant the proposed rights. UMI must review material restrictions before promotion. Dataset consent, privacy and third-party restrictions remain separate from permission to run inference.

Before contribution intake opens, publish the accepted-license list, exact contribution terms, review route and required provenance evidence. The signed policy binds the list and terms; matching a license identifier is not rights clearance. The [contributor review checklist](#) describes source lineage, permissions, reconstruction and decision records. Research-only, non-commercial or unspecified source permissions need explicit review; they are not automatically cleared by a new license on the bundle. Unresolved material restrictions hold promotion.

Preliminary review should let contributors resolve source questions before training. Record the disclosed versions, intended uses, policy, terms and any conditions. Reuse unchanged reviewed evidence at final review, while allowing reassessment for new facts or restrictions. Preliminary feedback does not guarantee promotion or replace review of the submitted artifact. Endpoint miners need not disclose private weights solely to serve requests, but remain responsible for applicable model, data and service-use restrictions.

Public baselines can support decentralized replication and independently operated products. Neither participation in Bittensor nor this endpoint track requires every miner to surrender private weights.

7 Reward calculation

Reward allocation is a required signed-policy value, expressed as integer basis points. There is no default allocation and this whitepaper does not activate a split. Endpoint and model-contribution shares sum to 10,000.

The approved initial launch allocation is 7,000 basis points (70%) for endpoint service and 3,000 (30%) for the current promoted model's contributor. Both reward tracks launch together. The signed activation policy must carry that allocation, and activation requires a qualifying preserved promotion with an eligible contributor. Approval of the allocation does not activate rewards or change the historical bootstrap.

For each closed round, qualifying endpoint submissions receive their share in proportion to their exact quality scores. Scores below the policy floor and invalid miner responses receive zero. A model-contribution share is paid to the registered contributor of the currently valid promoted baseline, provided it has qualifying fresh evaluation evidence in that round. Copying the public baseline does not transfer that attribution. The evaluator can run the preserved incumbent directly. Its contributor does not need to keep an inference server online or renew an old submission to make those preserved bytes available; current registration is still required.

This is king-of-the-hill attribution for the model contribution share, not winner-takes-all subnet emissions. A contributor may separately operate an endpoint and qualify for the service share. There are no automatic perpetual royalties after replacement.

If a nonzero track allocation has no qualifying recipient, the round cannot produce a successor row under the initial profile. It must report the reason and refrain from submission. It must not quietly donate the missing share, normalize it onto another track, fabricate scores or redirect it to an owner UID. A later policy may define a different explicit fallback.

Resolve recipients against a common finalized SN78 snapshot. Merge both track contributions by current UID, normalize the complete vector using exact rational arithmetic, and produce canonical u16 raw weights. Publish both the rational allocation and resulting raw row. Validators must recheck policy validity, registration and chain requirements before signing. Finalized consensus and incentive determine actual payouts.

A projected row is not a submitted or economically effective row. Observer status reports those states separately. The mechanism does not guarantee a particular amount of income.

8 Policy and state boundaries

All signed protocol objects use canonical JSON and domain-separated digests. A changed reward split, evaluation suite policy, resource profile, license policy or admission rule has a different policy identity.

Policy objects have a version, activation interval, predecessor identity and explicit track allocation. No implementation reads an arbitrary chain value into the policy and treats that as authorization. Chain requirements are checked against the approved policy.

Public state consists of submissions, acceptance receipts, closed round rosters, evaluation results, promotion records and projected or finalized weight evidence. Private state consists of wallets, sealed holdouts and operational credentials. Public status must not expose private state.

The registry and promotion store use durable transactions. A process crash cannot partially replace the current baseline or consume a valid retry. Evidence objects are content-addressed and immutable. The baseline pointer uses compare-and-swap against its recorded predecessor. Conflict evidence commits separately from the action it prevents, so an action rollback cannot remove the hold. Public status distinguishes historical promotion records from a baseline that is currently held for conflict.

Open submission is not open write access to the baseline registry. Only a verified promotion transaction may advance the baseline.

9 Threats and limitations

The initial successor must address:

- Sybil submissions and copied public models, through bounded intake, content identity and original promotion attribution;
- benchmark leakage and adaptive overfitting, through protected suites, fixed rosters, fresh paired evaluation and rate limits;
- malicious model files and dependencies, through byte verification, execution isolation and offline reconstruction;
- evaluator collusion, through disclosed control groups, independent agreement and replayable evidence;
- storage loss, through verified complete archives and restore tests;
- scheduler delay, through acceptance receipts, usable deadlines and explicit infrastructure failures; and
- policy or update-key compromise, through signed bounded releases, independent operator verification and a documented emergency hold.

These controls have limits. A signed license declaration can be false. Different hashes can represent copied behavior. A coordinator can censor intake. Evaluators can collude. Audit records expose many failures but do not make those trust assumptions disappear.

10 Release and activation

The implementation must first run without any weight-submit capability. A complete rehearsal exercises a new miner submission, replacement, paired evaluation, failure handling, archive restoration, promotion, baseline download by another miner, and exact row replay after restart.

Before activating successor rewards, publish:

- the signed policy, explicit reward allocation and activation interval;
- supported miner instructions and the complete admission interface;
- the initial preserved baseline and accepted license profile;
- the exact evaluator cohort, control-group disclosures and resource profile;
- evaluation-data rights and annotation provenance, dataset validation, split isolation and protected-suite procedure, without a mandatory human grading panel;
- successful endpoint and contributed-model end-to-end evidence, including signed run records from the selected evaluator cohort bound to the common result, and adversarial, timeout, restart, replay and failed-promotion cases;
- the deterministic row calculation and its finalized-chain preflight;

- the compatible signed validator release and tested upgrade path;
- the evidence cutoff, settlement record and late-conflict recovery rules; and
- monitoring, expiry, incident handling and rollback procedures.

The policy fixes the rehearsal duration and minimum evaluator evidence before it begins. Rehearsal output cannot be relabeled as production evidence. A change affecting scoring or trust invalidates the affected evidence and requires a new bounded rehearsal.

The current supervisor validates bootstrap input schemas on the host. Changing a worker image alone does not authorize a new manifest or profile. Test the actual installed supervisor against the successor. Publish a host upgrade only if required, with explicit operator consent and rollback. Never run competing weight writers for the same hotkey.

Keep the UID 0 and UID 54 registration-bridge processes operating within their signed policy until the approved transition or their submission deadline. A signed transition coordinates the stop of each old worker and activation of its successor, preserving authorization history, keys and transaction journals. The bridge's inherited hard sunset remains effective even if the successor is delayed.

11 Implementation status

This specification defines the successor target. It does not claim that open enrollment, arbitrary model evaluation, promotion or successor rewards are deployed. The repository's [implementation checklist](#) distinguishes tested code, local rehearsal tooling and remaining production integration.

The earlier transport, canonicalization, deterministic scoring, finality and supervisor components can be reused where their contracts match. Historical pilot success establishes neither a model contribution nor the successor's quality qualification.

References

- [UMI reference model](#)
- [ORO evaluation lifecycle](#)
- [Nous model contribution workflow](#)
- [CC BY-SA 4.0 terms](#)